CPE 334 — Project Deliverable 1

Engineering Baseline and Working Sprint 1

Semester: 1/2026 | **Due Date:** 8 November 2026 at 23:59 | **Grade Weight:** 80 points (8%)

Project & Submission Metadata

- **Theme / Selected Context:** University Department Workshop & Event Registration System (Theme: Event Registration and Attendance)
- **Repository URL:** https://github.com/ImanolLabarrere/A4D-Event-Registration
- **Submitted Release Tag:** `v0.1.0-sprint1` (commit `4d90108`)
- **Staging / Base Branches:** `sprint-1` merged into `main` via reviewed Pull Requests #23 and #24
- **Student Team:**
  - **Student A:** Xavier Gobert (`xgobert@et.esiea.fr` / GitHub: `hoshiiiix`)
  - **Student B:** Imanol Labarrere (Student ID: `69540460013` / GitHub: `ImanolLabarrere`)

Navigation Table (Deliverable Structure)

Section	Title	Content / Relevant IDs
Part A	SRS and Product Scope	Problem, Context, Actors, Scope/Exclusions, FR-01..12, BR-01..09, NFR-01..07, Decisions D-01..11, Assumptions A-01..07
Part B	Feature Inventory and Traceability	Features F-01..F-12, Cross-cutting R-01..R-04, Full Requirements-to-Feature-to-Sprint Traceability, Coverage gaps
Part C	System-Level SDS	Architecture, Modular Monolith Stack, Data Model & Prisma Schema, Migrations, UI screens, API conventions, Role matrix, Security & GCP Plan
Part D	Three-Sprint Plan	Sprints 1, 2, 3 goals, feature allocations, dependencies, completion outcomes, Sprint 1 feasible scope justification
Part E	Sprint 1 Engineering Contracts	Full specifications, UI/API/Data models, AC-101..609, STS TEST-101..610 for F-01, F-02, F-03, F-04, F-05, F-06 and Product DoD v1.0
Part F	Working Sprint 1 & Verification	Screen captures (normal, rejection, security), Test results (101 Vitest + 3 E2E Playwright), Code excerpts (TEST-411, TEST-603/604), TDD Red/Green cycles, PR evidence (#15, #20), Contribution table, AI declaration, Setup summary

---

Part A: Software Requirements Specification (SRS) & Product Scope

(Score: 10 points)

SRS — Event Registration and Attendance System (v1.0)

1. Context and problem

A university department runs workshops and talks with limited seating. Registrations are currently handled through messages or spreadsheets, which leads to duplicate sign-ups, overbooking, and no reliable view for organizers. The product lets organizers publish events with a limited capacity and lets participants register for them, with strict capacity control. Later sprints add cancellation, a waiting list, and attendance tracking.

2. Actors

Actor	Responsibilities
Organizer	Creates and publishes their own events, views the registrations of their own events
Participant	Browses published events, registers, views their own registrations

3. Scope and exclusions

Included: authentication, events, registrations, cancellation, waiting list, check-in, attendance reports. Excluded: payments, emails and notifications, self-registration of accounts, password recovery, multiple roles per account, recurring events.

4. Functional requirements

ID	Requirement	Sprint
FR-01	A user logs in, logs out, and retrieves their current identity	1
FR-02	An organizer creates an event (title, date, capacity) as a draft	1
FR-03	An organizer publishes their own event	1

ID	Requirement	Sprint
FR-04	A participant lists published events	1
FR-05	A participant registers for a published event	1
FR-06	A participant views their own registrations; an organizer views the registrations of their own event	1
FR-07	A participant cancels their registration	2
FR-08	A participant is placed on a waiting list when the event is full, then promoted in order of arrival	2
FR-09	An organizer edits their event (title, date, capacity)	2
FR-10	A registration deadline can be set per event	2
FR-11	An organizer records participant check-in	3
FR-12	An organizer views an attendance report	3

5. Business rules

ID	Rule	Sprint
BR-01	Capacity is a strictly positive integer	1
BR-02	An event can only be published if its date is in the future	1
BR-03	A participant has at most one registration per event, permanently: no re-registration after cancellation	1
BR-04	The number of confirmed registrations never exceeds the capacity, even with simultaneous requests	1
BR-05	Registration for a past event is not allowed	1
BR-06	An unpublished event is invisible to participants	1
BR-07	An organizer can only manage their own events	1

ID	Rule	Sprint
BR-08	The waiting list is limited to floor(20% of capacity); an event with fewer than 5 places has no waiting list	2
BR-09	A participant can only see their own registrations	1

6. Non-functional requirements

ID	Requirement and measurable target	Verification method	Established / verified
NFR-01	Any request to a protected endpoint without a valid session returns 401 and a body that contains no private data (no draft, no registration, no user data)	API test calling every protected endpoint without a cookie	Sprint 1 / Sprint 1, re-run in each later sprint
NFR-02	Passwords are stored only as argon2id hashes; no plaintext password exists in the database	Integration test: the stored hash starts with `$argon2id$` and differs from the password	Sprint 1 / Sprint 1
NFR-03	With 20 simultaneous registration requests from 20 distinct participants for the last remaining place, exactly 1 request succeeds (201), 19 receive 409 EVENT_FULL, and confirmed registrations equal the capacity	Integration test on the real PostgreSQL database with `Promise.all` (no mock)	Sprint 1 / Sprint 1
NFR-04	Data survives a restart: the number of events and registrations after restarting the API and the database container equals the number before	Documented manual procedure with before/after counts (Sprint 1); repeated after redeployment on GCP (Lab 5)	Sprint 1 / Sprint 1 and Lab 5
NFR-05	No secret is stored in Git: `.env` is not tracked, `.env.example` contains placeholders only	Release check: `git ls-files` lists no `.env`; manual review of `.env.example`	Sprint 1 / Sprint 1
NFR-06	A session expires after 8 hours; an expired or logged-out session cookie is rejected with 401	Integration test: set `expiresAt` in the past, then call `/api/auth/me`; call `/api/auth/me` with the cookie of a logged-out session	Sprint 1 / Sprint 1
NFR-07	Error responses never contain a stack trace, SQL text or password hash	API test: provoke validation and server errors and inspect the response bodies	Sprint 1 / Sprint 3 regression

7. Assumptions, decisions, questions

ID	Decision	Rationale
D-01	No re-registration after cancellation	UNIQUE(event, user) constraint across all statuses
D-02	Waiting list = 20% of places, rounded down, in Sprint 2	Promotion requires cancellation (Sprint 2)
D-03	Registration is rejected if the event is in the past	Business consistency
D-04	An organizer manages only their own events	Default option of the brief
D-05	A draft event is invisible to participants (404)	Avoids information leakage
D-06	Server-side sessions, httpOnly cookie, 8 h expiry	Logout truly revokes the session
D-07	Full event in Sprint 1: 409 EVENT_FULL response	Waiting list deferred to Sprint 2

Open questions:

ID	Question	Resolution
Q-01	What happens if an organizer reduces capacity below the number of registered participants?	To be decided before Sprint 2 (proposal: reject the reduction)

Additional assumptions (B)

ID	Assumption
A-01	All dates are stored in UTC and displayed in the user's local time
A-02	An event has exactly one organizer; co-organizers are out of scope
A-03	Accounts are created by the seed only; there is no self-registration
A-04	Event title is 1 to 120 characters; capacity is an integer from 1 to 1000
A-05	The email is the login identifier and is compared in lower case
A-06	In Sprint 1 a published event cannot be unpublished
A-07	All demo data is synthetic

Additional decisions (B)

ID	Decision	Rationale
D-08	Error convention: 401 when not authenticated, 403 when the role is not allowed, 404 when the resource does not exist or does not belong to the caller (another participant's registration, a draft seen by a participant, another organizer's event), 409 for state conflicts (EVENT_FULL, ALREADY_REGISTERED, EVENT_IN_PAST, ALREADY_PUBLISHED), 400 for invalid input	One rule for every contract; a 404 does not reveal that a private resource exists
D-09	A failed login always returns the same message and 401, whether the email or the password is wrong	Does not reveal which emails exist
D-10	Only the PARTICIPANT role can register; an organizer who tries receives 403	Matches the permission matrix and the stakeholder brief
D-11	The event list requires authentication; anonymous users cannot list events	Matches the permission matrix and NFR-01

Part B: Feature Inventory and Traceability

(Score: 10 points)

Feature Inventory (v1.0)

Every functional requirement (FR), business rule (BR) and non-functional requirement (NFR) of the SRS is allocated to a feature or to a named system responsibility.

1. Features

ID	Feature	Description	Requirements	Depends on	Sprint
F-01	Authentication	Login with email and password, logout, current user, server-side role checks	FR-01, NFR-01, NFR-02, NFR-06	none	1
F-02	Create and publish an event	An organizer creates a draft (title, date, capacity) and publishes their own event if its date is in the future	FR-02, FR-03, BR-01, BR-02, BR-07	F-01	1
F-03	Browse published events	An authenticated user lists and opens published events; drafts are invisible to participants	FR-04, BR-06	F-01, F-02	1

ID	Feature	Description	Requirements	Depends on	Sprint
F-04	Register for an event	A participant registers for a published event; capacity, duplicate and past-event rules; safe under concurrent requests	FR-05, BR-03, BR-04, BR-05, NFR-03	F-01, F-02, F-03	1
F-05	My registrations	A participant lists and opens their own registrations only	FR-06, BR-09	F-04	1
F-06	Organizer view of registrations	An organizer lists the registrations of their own event	FR-06, BR-07	F-02, F-04	1
F-07	Cancel a registration	A participant cancels their registration; the place is released	FR-07, BR-03	F-04	2
F-08	Waiting list	When an event is full, a participant joins a waiting list limited to floor(20% of capacity); promotion in order of arrival	FR-08, BR-08	F-07	2
F-09	Edit an event	An organizer edits title, date and capacity of their own event (question Q-01 on capacity reduction)	FR-09, BR-01, BR-07	F-02	2
F-10	Registration deadline	An organizer sets a deadline per event; registration after it is rejected	FR-10	F-02, F-04	2
F-11	Check-in	An organizer records the check-in of a registered participant	FR-11, BR-07	F-04	3
F-12	Attendance report	An organizer views the attendance report of their own event	FR-12, BR-07	F-11	3

2. Cross-cutting system responsibilities

ID	Responsibility	Requirements	Established and verified
R-01	Backend authorization (session check, role check, ownership check) applied to every protected endpoint	NFR-01, BR-07, BR-09	Established in F-01 (Sprint 1), re-verified for each new endpoint
R-02	Database integrity: constraints and migrations	BR-01, BR-03, BR-04	Sprint 1
R-03	Reproducible setup: seed data, environment configuration, no secrets in Git	NFR-04, NFR-05	Sprint 1

ID	Responsibility	Requirements	Established and verified
R-04	Safe error handling: uniform error format, no internal details leaked	NFR-07	Sprint 1, regression in Sprint 3

3. Coverage check

Requirement	Covered by
FR-01	F-01
FR-02, FR-03	F-02
FR-04	F-03
FR-05	F-04
FR-06	F-05 (participant), F-06 (organizer)
FR-07	F-07
FR-08	F-08
FR-09	F-09
FR-10	F-10
FR-11	F-11
FR-12	F-12
BR-01	F-02, F-09, R-02
BR-02	F-02
BR-03	F-04, F-07, R-02
BR-04	F-04, R-02
BR-05	F-04
BR-06	F-03
BR-07	F-02, F-06, F-09, F-11, F-12, R-01
BR-08	F-08
BR-09	F-05, R-01
NFR-01	R-01 (all features)
NFR-02	F-01
NFR-03	F-04, R-02
NFR-04	R-03

Requirement	Covered by
NFR-05	R-03
NFR-06	F-01
NFR-07	R-04

Uncovered requirements: none. Approved changes: none at this version.

Traceability Matrix

Requirements Traceability (v1.0)

Each Sprint 1 feature owns a range of acceptance criteria and tests: F-01 AC-1xx/TEST-1xx, F-02 AC-2xx/TEST-2xx, F-03 AC-3xx/TEST-3xx, F-04 AC-4xx/TEST-4xx, F-05 AC-5xx/TEST-5xx, F-06 AC-6xx/TEST-6xx. Exact IDs are in the feature contracts (docs/sprints/sprint-1/). Features of Sprints 2 and 3 get their AC and tests in their own contract before implementation.

1. Functional requirements

Requirement	Feature	Sprint	Verification
FR-01	F-01 Authentication	1	AC-101..114 / TEST-101..114 (contract F-01)
FR-02	F-02 Create and publish an event	1	AC-201..203, 213 / TEST-203..205, 215, 216
FR-03	F-02 Create and publish an event	1	AC-206..211 / TEST-208..213, 216
FR-04	F-03 Browse published events	1	AC-301..313 / TEST-301..314 (contract F-03)
FR-05	F-04 Register for an event	1	AC-401..412 / TEST-401..414
FR-06	F-05 My registrations; F-06 Organizer view	1	AC-501..508 / TEST-501..509; AC-601..609 / TEST-601..610
FR-07	F-07 Cancel a registration	2	Contract at Sprint 2
FR-08	F-08 Waiting list	2	Contract at Sprint 2
FR-09	F-09 Edit an event	2	Contract at Sprint 2
FR-10	F-10 Registration deadline	2	Contract at Sprint 2
FR-11	F-11 Check-in	3	Contract at Sprint 3
FR-12	F-12 Attendance report	3	Contract at Sprint 3

2. Business rules

Requirement	Feature or responsibility	Sprint	Verification
BR-01	F-02, R-02	1	AC-202, AC-212 / TEST-201, 204, 214
BR-02	F-02	1	AC-207 / TEST-202, 209
BR-03	F-04, R-02 (extended by F-07 in Sprint 2)	1	AC-403, AC-410, AC-411 / TEST-401, 405, 412, 413
BR-04	F-04, R-02	1	AC-402, AC-409, AC-412 / TEST-401, 403, 404, 411
BR-05	F-04	1	AC-404 / TEST-401, 406
BR-06	F-03; F-04 (draft returns 404)	1	AC-302..305 / TEST-302, 305, 306; AC-405 / TEST-407
BR-07	F-02, F-06, R-01	1	AC-209 / TEST-211; AC-601..607 / TEST-601..607
BR-08	F-08	2	Contract at Sprint 2
BR-09	F-05, R-01	1	AC-501, AC-504, AC-505 / TEST-501, 504, 505

3. Non-functional requirements

Requirement	Feature or responsibility	Sprint	Verification
NFR-01	R-01 (all protected endpoints)	1	AC-106, 111, AC-205, 210, AC-306, 308, AC-408, AC-506, 507, AC-604, 605 / TEST-102, 109, 207, 212, 307, 309, 410, 506, 507, 604, 605
NFR-02	F-01	1	AC-110 / TEST-112 (argon2id hash check)
NFR-03	F-04, R-02	1	AC-409 / TEST-411 (real database, 20 concurrent requests)
NFR-04	R-03	1	Manual before/after count procedure, documented in Part F
NFR-05	R-03	1	Release check: git ls-files shows no .env
NFR-06	F-01	1	AC-108, 109 / TEST-103, 111 (8h TTL, delete on expiry)
NFR-07	R-04	1	AC-508 / TEST-508; error-body checks in each feature contract

4. Coverage gaps

None: every FR, BR and NFR has a feature or system responsibility and a sprint. All Sprint 1 AC and test IDs are fully mapped and verified.

5. Approved changes

None at this version.

Part C: System-Level Software Design Specification (SDS)

(Score: 15 points)

System-Level SDS (v1.0)

1. Architecture and stack

Modular monolith: UI (React/Vite) → REST API (Express/TypeScript) → PostgreSQL (Prisma).

- UI: display and forms, no security rules.
- API: authentication, authorization, validation (zod), business rules BR-01 to BR-09.
- Database: integrity constraints (UNIQUE, CHECK capacity > 0, foreign keys). Communication: HTTP/JSON, httpOnly session cookie. Critical rules (capacity, duplicates) are enforced server-side inside a transaction using SELECT ... FOR UPDATE on the event row, and backed by database constraints.

2. Data model

Entity	Main fields	Constraints
User	id, email, passwordHash, role	unique email; role ∈ {PARTICIPANT, ORGANIZER}
Session	id (random token), userId, expiresAt	cascade delete with the user
Event	id, title, startsAt, capacity, status, organizerId	capacity > 0; status ∈ {DRAFT, PUBLISHED}; organizerId → User
Registration	id, eventId, userId, status, createdAt	UNIQUE(eventId, userId); status = CONFIRMED (Sprint 1)

Ownership: an Event belongs to its organizer; a Registration belongs to its participant.

```
erDiagram
  USER ||--o{ SESSION : has
  USER ||--o{ EVENT : organizes
  USER ||--o{ REGISTRATION : makes
  EVENT ||--o{ REGISTRATION : receives
```

3. Migrations and demo data

Prisma migrations (`npx prisma migrate dev`). Reproducible seed (`npm run db:seed`): 2 organizers, 3 participants, 1 published event and 1 draft event per organizer. Demo passwords are read from the environment and never committed.

4. UI, API, roles, authentication, configuration

4.1 Screens and navigation

Screen	Role	Content
Login	anonymous	Email and password form
Event list	participant, organizer	Published events with title, date, capacity and remaining places
Event detail	participant	Event information and a Register button
My registrations	participant	The participant's own registrations
My events	organizer	The organizer's events, including drafts, with Create and Publish actions
Event registrations	organizer	Registrations of one of the organizer's events

Navigation depends on the role returned by `GET /api/auth/me`. This only adapts the display: the backend enforces every permission.

Shared UI behavior:

- Loading: a visible loading indicator while a request is pending; buttons are disabled to prevent double submission.
- Empty state: an explicit message when a list is empty (for example "No registration yet").
- Error state: the message of the API error is shown (EVENT_FULL, ALREADY_REGISTERED, validation errors) with a way to retry.
- Validation: the form shows field errors returned by the API; client-side checks are a convenience, never the rule.
- An expired session (401) sends the user to the Login screen.

4.2 API conventions

- Base path `/api`, JSON requests and responses, resources named in the plural.
- Input is validated with zod; invalid input returns 400.
- The identity and role of the caller always come from the server-side session, never from the request body, a header or a query parameter.

- Error body, identical for all errors: `{ "error": { "code": "EVENT_FULL", "message": "The event is full" } }`. Error responses never include a stack trace, SQL or internal identifiers beyond the requested resource.
- Status codes follow decision D-08 (401, 403, 404, 409, 400).

Endpoints of Sprint 1 (details are given in each feature contract):

Method and path	Feature	Success
POST `/api/auth/login`	F-01	200, user and session cookie
POST `/api/auth/logout`	F-01	204
GET `/api/auth/me`	F-01	200, current user
POST `/api/events`	F-02	201, draft event
POST `/api/events/:id/publish`	F-02	200, published event
GET `/api/events`	F-03	200, published events
GET `/api/events/mine`	F-02, F-03	200, the organizer's events including drafts
GET `/api/events/:id`	F-03	200, event
POST `/api/events/:id/registrations`	F-04	201, registration
GET `/api/registrations/mine`	F-05	200, own registrations
GET `/api/registrations/:id`	F-05	200, own registration
GET `/api/events/:id/registrations`	F-06	200, registrations of the organizer's event

4.3 Role and permission matrix

Operation	Anonymous	Participant	Organizer
Log in	yes	yes	yes
Log out, get current user	no	yes	yes
List published events, open a published event	no	yes	yes
Open a draft event	no	no (404)	own events only
List own events including drafts	no	no (403)	yes
Create an event	no	no (403)	yes
Publish an event	no	no (403)	own events only (404 otherwise)
Register for an event	no	yes	no (403)

Operation	Anonymous	Participant	Organizer
List own registrations, open one registration	no	own only (404 otherwise)	no (403)
List the registrations of an event	no	no (403)	own events only (404 otherwise)

Ownership links: an Event belongs to its organizer (`organizerId`); a Registration belongs to its participant (`userId`).

4.4 Authentication and session lifecycle

- Accounts are created by the seed. Passwords are hashed with argon2id; plaintext passwords are never stored or logged.
- Login: the server checks the email and password, then creates a row in the `Session` table with a random token (32 random bytes, hexadecimal) and `expiresAt = now + SESSION_TTL_HOURS` (8 hours).
- The token is sent in a cookie named `sid` with `HttpOnly` , `SameSite=Lax` , `Path=/` , and `Secure` when `NODE_ENV=production` . The token is never readable by client JavaScript.
- Every protected request: the middleware reads the cookie, loads the session and its user, rejects with 401 if the session is missing or expired (and deletes an expired session), then attaches the user and role to the request.
- Current user: `GET /api/auth/me` returns the id, email and role of the session's user.
- Logout: the session row is deleted and the cookie cleared, so the old cookie is rejected afterwards (NFR-06). Logout is idempotent and returns 204.
- A failed login returns 401 with the same message whatever the cause (D-09).

4.5 Configuration

Variable	Purpose	Example in `.env.example`
DB_PASSWORD	Password of the local PostgreSQL container	`change-me`
DATABASE_URL	Connection string of the application database	`postgresql://a4d:change-me@localhost:5433/a4d`
DATABASE_URL_TEST	Connection string of the test database	`postgresql://a4d:change-me@localhost:5433/a4d_test`
SESSION_TTL_HOURS	Session lifetime	`8`
SEED_PASSWORD	Password given to the demo accounts by the seed	`change-me-demo`

- The local PostgreSQL container is published on host port 5433, so it does not clash with a PostgreSQL already installed on port 5432. Inside the container PostgreSQL still listens on 5432.

- `.env` is ignored by Git; only `.env.example`, with placeholder values, is committed (NFR-05).
- In deployment, values come from the platform's environment and Secret Manager, never from the repository or from client code.
- The migration `init` creates the schema and the constraints `capacity_positive` (CHECK capacity > 0) and the UNIQUE index on (eventId, userId).

5. GCP deployment plan

Target: Cloud Run (application container), Cloud SQL PostgreSQL (durable storage), Secret Manager (secrets). Docker image built in Lab 5. Environment variables: DATABASE_URL, SESSION_TTL_HOURS, SEED_PASSWORD. No secrets in Git or in client code.

Part D: Three-Sprint Plan

(Score: 5 points)

Three-Sprint Plan

Sprint	Goal	Features	Dependencies	Expected outcome
1 (8 Nov)	Vertical slice with authentication, permissions, persistence	F-01 Auth, F-02 Events, F-03 Browse, F-04 Registration, F-05 My registrations, F-06 Organizer view	None	Complete organizer then participant journey, rules and access restrictions tested
2 (29 Nov)	Core expansion	Cancellation, 20% waiting list, event editing, registration deadline	Sprint 1	Full registration lifecycle
3 (13 Dec)	Completion and hardening	Check-in, attendance, reports, hardening, regression	Sprint 2	Complete and verified product

Sprint 1 scope

Feasible in three days: six short features, a single flow, and one tricky rule (concurrency). The waiting list and cancellation are deferred because they depend on each other.

Part E: Sprint 1 Engineering Contracts & Definition of Done

(Score: 20 points)

> This section provides the complete, unabridged engineering contracts prepared before implementation for all Sprint 1 features, establishing the feature SDS, applicable UI/API/Data specifications, acceptance criteria (AC), and Software Test Specification (STS).

1. Product Definition of Done (DoD v1.0)

Product Definition of Done — v1.0

A Sprint 1 feature is done only when all points are met and supported by evidence (not by an AI agent's statement):

1. Required behavior is implemented as described in its contract (UI, API and data where applicable).
2. Every planned test of the STS is implemented and executed; each AC maps to at least one passing automated test, and negative, boundary and role cases are included.
3. For rule work, the intended-behavior test was seen failing before the implementation and passing after (output saved in docs/evidence/).
4. Authorization is enforced by the backend; no client-selected user id or role is trusted.
5. Error responses follow D-08 and contain no stack trace, SQL or password hash.
6. The change was made on a feature branch from sprint-1, reviewed by the other student in a PR with a specific check or finding, and merged into sprint-1; authors never approve their own PR.
7. Integration tests and acceptance checks pass on sprint-1; the reviewed release PR sprint-1 to main is merged and tagged (v0.1.0-sprint1).
8. README, contracts, traceability and .env.example are current; no secret is committed.
9. Test command, counts (passed, failed, skipped), and the tested commit are recorded. Failed or skipped tests and incomplete work are disclosed; no required test is disabled or weakened.

---

2. Engineering Contract F-01: Authentication

F-01 Authentication — v1.0

Requirements: FR-01, NFR-01, NFR-02, NFR-06, NFR-07, R-01, R-04 | Depends on: none | Owner: B | Reviewer: A

1. Behavior and workflow

A user logs in with email and password. The server creates a session row with a random token and sends it in an httpOnly cookie. Every protected request is authenticated from that cookie. The user can retrieve their identity and log out, which deletes the session. Accounts come from the seed (no self-registration, A-03). The feature also provides the shared middlewares requireAuth and requireRole(...roles) used by all later features (R-01).

2. Business rules, validation, failures

- email: string, trimmed and lower-cased before lookup, 1 to 254 characters (A-05). password: string, 1 to 200 characters. Anything else returns 400 VALIDATION_ERROR.
- A wrong password and an unknown email return the same 401 INVALID_CREDENTIALS with the same body (D-09). No cookie is set on failure.
- Token: 32 random bytes from a cryptographic generator, hexadecimal (64 characters). Session lifetime: SESSION_TTL_HOURS (8 hours, NFR-06).
- Cookie: name sid, HttpOnly, SameSite=Lax, Path=/, Secure when NODE_ENV=production.
- requireAuth: missing, unknown or expired cookie returns 401 UNAUTHENTICATED; an expired session row is deleted. requireRole: authenticated but role not allowed returns 403 FORBIDDEN.
- Identity and role come only from the session. Any userId, role or similar value in the body, query or headers is ignored.
- Logout deletes the session row and clears the cookie; it is idempotent (204 even without a session).
- Passwords are stored only as argon2id hashes (NFR-02). A passwordHash is never returned or logged.
- Error body follows D-08 and SDS 4.2: { "error": { "code", "message" } }, with no stack trace or SQL (NFR-07).

3. Permissions

Endpoint	Anonymous	Participant	Organizer
POST /api/auth/login	allowed	allowed	allowed
POST /api/auth/logout	204	204	204
GET /api/auth/me	401	200	200

4. UI

Login page: email and password fields, Log in button disabled while the request runs (loading state), one generic error message on 401, field messages on 400. After login the navigation depends on the role returned by /api/auth/me (participant: events and My registrations; organizer: My events). A Log out button is always visible when logged in. When any request returns 401, the user is sent back to the Login page. Hiding navigation is cosmetic; the backend enforces every permission.

5. API

POST /api/auth/login — body: { "email": string, "password": string }

- 200: { "id", "email", "role" } and Set-Cookie: sid=...
- 400 VALIDATION_ERROR, 401 INVALID_CREDENTIALS.

POST /api/auth/logout — no body

- 204 and a cleared sid cookie.

GET /api/auth/me — no body

- 200: { "id", "email", "role" }
- 401 UNAUTHENTICATED.

Shared error codes: UNAUTHENTICATED (401), FORBIDDEN (403), NOT_FOUND (404), VALIDATION_ERROR (400).

6. Data

Tables User and Session (migration init, no new migration). Session: id (token), userId, expiresAt, cascade delete with the user. No schema change.

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-101	Login with a valid email and password	200 with id, email, role; sid cookie set; a Session row exists for the user
AC-102	Login with a wrong password	401 INVALID_CREDENTIALS; no cookie; no Session row created
AC-103	Login with an unknown email	Same status and same body as AC-102
AC-104	Login with a missing, empty or non-string email or password	400 VALIDATION_ERROR; no Session row created
AC-105	GET /api/auth/me with a valid cookie	200 with the identity and role of the session user
AC-106	GET /api/auth/me without a cookie	401 UNAUTHENTICATED; no private data in the body
AC-107	Logout, then reuse the old cookie	Logout 204 and the Session row is deleted; the old cookie gets 401 on /api/auth/me; a second logout also returns 204

AC	Criterion	Observable outcome
AC-108	Session whose expiresAt is in the past	401 on /api/auth/me and the expired row is deleted
AC-109	Session cookie and token properties	Cookie is HttpOnly, SameSite=Lax, Path=/; token is 64 hexadecimal characters; expiresAt is about 8 hours after login
AC-110	Password storage and responses	Stored passwordHash starts with $argon2id$ and differs from the password; no response contains passwordHash
AC-111	Role check middleware	Unauthenticated request gets 401; authenticated request with a role outside the allowed list gets 403; allowed role passes
AC-112	Request carries userId, role, X-User-Role header or a query parameter	Ignored: /api/auth/me still returns the session user's own identity
AC-113	Login with the email in upper case or with surrounding spaces	Succeeds as for the lower-case email
AC-114	Garbage or unknown cookie value	401 UNAUTHENTICATED

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-101	AC-109	Unit	Function generateSessionToken	Generate 1000 tokens	Each is 64 hexadecimal characters; all distinct
TEST-102	AC-111	Unit	requireRole with fake request and response objects	No user; user with a non-allowed role; user with an allowed role	401; 403; next() called
TEST-103	AC-101, AC-109	API/DB	Seeded organizer	POST login with valid credentials	200 with id, email, role; Set-Cookie has sid, HttpOnly, SameSite=Lax, Path=/; Session row exists with expiresAt within 8 hours (tolerance 1 minute)
TEST-104	AC-102	API/DB	Seeded organizer, Session count N	POST login with a wrong password	401 INVALID_CREDENTIALS; no Set-Cookie; count still N
TEST-105	AC-103	API	Unknown email	POST login	Same status and body as TEST-104
TEST-106	AC-104	API/DB	Session count N	POST login with no body, empty strings, numbers	400 VALIDATION_ERROR each; count still N

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-107	AC-113	API	Seeded participant	POST login with upper-case email and spaces around it	200
TEST-108	AC-105	API	Logged-in participant	GET /api/auth/me with the cookie	200; id, email and role match the seeded user
TEST-109	AC-106, AC-114	API	No cookie; cookie "sid=garbage"	GET /api/auth/me	401 both; body has no email, role or id
TEST-110	AC-107	API/DB	Logged-in user	POST logout, GET /api/auth/me with the old cookie, POST logout again	204; Session row deleted; 401; 204
TEST-111	AC-108	API/DB	Session whose expiresAt is set in the past	GET /api/auth/me	401; Session row deleted
TEST-112	AC-110	DB/API	Seeded users	Read User.passwordHash from the database; inspect login and me bodies	Starts with $argon2id$; differs from SEED_PASSWORD; no passwordHash key in any response
TEST-113	AC-112	API	Logged-in participant	GET /api/auth/me with header X-User-Role: ORGANIZER and query ?userId=other	Identity and role remain those of the participant
TEST-114	AC-101, AC-105, AC-107	E2E	Seeded organizer	Log in through the UI, see the organizer navigation, log out, reload; then call the API with the old cookie	Login page shown after logout and reload; API returns 401

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: test output, tested commit, TEST-to-file mapping, red then green evidence (docs/evidence/F-01-red.txt, F-01-green.txt) and the TEST-110 and TEST-112 results as proof for NFR-06 and NFR-02.

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	B	Initial contract, written before implementation

---

3. Engineering Contract F-02: Create and Publish Events

F-02 Create and publish an event — v1.0

Requirements: FR-02, FR-03, BR-01, BR-02, BR-07, NFR-01, NFR-07, R-01, R-02 | Depends on: F-01 | Owner: A | Reviewer: B

1. Behavior and workflow

An authenticated organizer creates an event in DRAFT status (title, start date, capacity). The event belongs to the organizer identified by the session. The organizer then publishes it. Only published events are visible to and registrable by participants (BR-06, verified in F-03 and F-04). Publishing is one-way in Sprint 1 (no unpublish).

2. Business rules, validation, failures

- title: string, 1 to 120 characters after trimming.
- startsAt: valid ISO 8601 date-time. Any date is accepted at creation; BR-02 is checked at publication.
- capacity: integer, 1 to 10000 (BR-01; the upper bound is an assumption recorded here). Zero, negative, decimal, string or missing values return 400 VALIDATION_ERROR.
- Fields id, status and organizerId sent in the body are ignored; the owner is always the session user.
- Publish checks, in this order: event exists and belongs to the caller (else 404); status is DRAFT (else 409 ALREADY_PUBLISHED); startsAt is in the future (else 409 EVENT_IN_PAST).
- Error body follows D-08 and SDS 4.2: { "error": { "code", "message" } }. No stack trace or SQL.

3. Permissions

Caller	POST /api/events	POST /api/events/:id/publish
Anonymous	401	401
Participant	403	403
Organizer, owner	201	200
Organizer, not owner	n/a	404

4. UI

Organizer page "New event": form with title, date-time and capacity; Create button disabled while the request runs (loading state); field-level messages for 400. After creation the draft is shown with a Publish button; after publication a "Published" badge replaces it. A 409 is shown as an error banner with the server message. Hiding the page from participants is cosmetic: the backend enforces the role.

5. API

POST /api/events — body: { "title": string, "startsAt": ISO date-time, "capacity": integer }

- 201: { "id", "title", "startsAt", "capacity", "status": "DRAFT", "organizerId" }
- 400 VALIDATION_ERROR, 401, 403.

POST /api/events/:id/publish — no body

- 200: same event shape with "status": "PUBLISHED"
- 401, 403, 404 (absent, malformed id, or not owned), 409 ALREADY_PUBLISHED, 409 EVENT_IN_PAST.

6. Data

Table Event (migration init, no new migration): id, title, startsAt, capacity, status (DRAFT/PUBLISHED), organizerId. Constraint capacity_positive (CHECK capacity > 0).

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-201	Organizer creates a valid event	201, status DRAFT, organizerId = caller, event retrievable from the database
AC-202	Capacity 0, -1, 1.5, "10" or missing	400 VALIDATION_ERROR, nothing stored
AC-203	Empty title or invalid date	400 VALIDATION_ERROR, nothing stored
AC-204	Participant creates an event	403, nothing stored
AC-205	Anonymous creates an event	401, no private data in the body
AC-206	Organizer publishes their own draft with a future date	200, status PUBLISHED, persisted
AC-207	Publish an event whose date is not in the future	409 EVENT_IN_PAST, status stays DRAFT

AC	Criterion	Observable outcome
AC-208	Publish an already published event	409 ALREADY_PUBLISHED, no change
AC-209	Organizer publishes another organizer's event	404, status unchanged
AC-210	Participant publishes an event	403, status unchanged
AC-211	Publish a non-existent or malformed id	404
AC-212	Direct database insert with capacity <= 0	Rejected by constraint capacity_positive
AC-213	Body contains organizerId or status PUBLISHED	Ignored: event is DRAFT and owned by the caller

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-201	AC-202	Unit	Schema createEventSchema	Validate capacity 0, -1, 1.5, "10", missing	Each rejected; capacity 1 and 10000 accepted
TEST-202	AC-207	Unit	Function canPublish(event, now)	now equal to or after startsAt; now one second before	false, then true
TEST-203	AC-201	API/DB	Logged-in organizer	POST valid event	201 DRAFT; row exists with organizerId
TEST-204	AC-202	API/DB	Logged-in organizer, event count N	POST capacity 0	400; count still N
TEST-205	AC-203	API	Logged-in organizer	POST empty title; POST "not-a-date"	400 both
TEST-206	AC-204	API/DB	Logged-in participant	POST valid event	403; count unchanged
TEST-207	AC-205	API	No cookie	POST valid event	401; body has no private data
TEST-208	AC-206	API/DB	Own draft, future date	POST publish	200; row status PUBLISHED
TEST-209	AC-207	API/DB	Own draft, past date	POST publish	409 EVENT_IN_PAST; status DRAFT

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-210	AC-208	API	Own published event	POST publish	409 ALREADY_PUBLISHED
TEST-211	AC-209	API/DB	Draft owned by organizer 2, caller organizer 1	POST publish	404; status DRAFT
TEST-212	AC-210	API/DB	Logged-in participant, existing draft	POST publish	403; status DRAFT
TEST-213	AC-211	API	Logged-in organizer	POST publish unknown id and "abc"	404 both
TEST-214	AC-212	DB	Direct SQL insert	Insert capacity 0	Constraint violation
TEST-215	AC-213	API/DB	Logged-in organizer	POST with organizerId of another user and status PUBLISHED	201; DRAFT; organizerId = caller
TEST-216	AC-201, AC-206	E2E	Seeded organizer	Create then publish through the UI	Draft then Published badge; event persists after reload

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: test output, tested commit, TEST-to-file mapping, red then green evidence (docs/evidence/F-02-red.txt, F-02-green.txt).

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	A	Initial contract, written before implementation

---

4. Engineering Contract F-03: Browse Published Events

F-03 Browse published events — v1.0

Requirements: FR-04, BR-06, BR-07, NFR-01, NFR-07, R-01 | Depends on: F-01, F-02 | Owner: B | Reviewer: A

1. Behavior and workflow

An authenticated user lists upcoming published events and opens one. A participant never sees a draft (BR-06): a draft behaves exactly like a non-existent event. An organizer additionally lists their own events, drafts included, and can open their own drafts. Each event shows how many places remain, computed from the confirmed registrations.

2. Business rules, validation, failures

- GET /api/events returns only events with status PUBLISHED and startsAt in the future, ordered by startsAt ascending. Past published events are not listed (decision recorded here) but can still be opened by id.
- GET /api/events/mine returns every event owned by the caller, drafts included, ordered by startsAt ascending. Only organizers may call it.
- GET /api/events/:id: a PUBLISHED event is returned to any authenticated user. A DRAFT event is returned only to its organizer; for anyone else it is a 404 identical to an unknown id (D-05, D-08). An unknown or malformed id is a 404.
- confirmedCount is the number of CONFIRMED registrations; remainingPlaces = max(0, capacity - confirmedCount). Both are computed from the Registration table; no counter is stored.
- Responses contain no emails, password hashes, organizer identity or registration data, only the counts (NFR-07).
- The route /api/events/mine must be declared before /api/events/:id.

3. Permissions

Caller	GET /api/events	GET /api/events/mine	GET /api/events/:id
Anonymous	401	401	401
Participant	200 published	403	200 if published; 404 if draft or unknown
Organizer	200 published	200 own events	200 if published or own draft; 404 for another organizer's draft or unknown

4. UI

Event list page for participants and organizers: loading indicator, empty state ("No upcoming events"), error state with retry, and one row per event with title, date and remaining places. A row opens the event detail page (title, date, capacity, remaining places; the Register button belongs to F-04). Organizer page "My events": same states, with a Draft or Published badge per event; Create and Publish actions belong to F-02. A 404 on the detail page shows "Event not found".

5. API

GET /api/events — 200: [{ "id", "title", "startsAt", "capacity", "confirmedCount", "remainingPlaces", "status" }] GET /api/events/mine — 200: same shape. GET /api/events/:id — 200: same shape; 404 NOT_FOUND. All: 401 UNAUTHENTICATED; /mine: 403 FORBIDDEN for a participant.

6. Data

No schema change. Read-only on Event with a count of CONFIRMED Registration rows.

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-301	A participant lists events	Only PUBLISHED events with a future date, ordered by date ascending
AC-302	A draft event exists	It never appears in GET /api/events, for a participant or an organizer
AC-303	A participant opens a draft event by id	404, no event data in the body
AC-304	An organizer opens another organizer's draft	404, no event data in the body
AC-305	An organizer opens their own draft	200 with the event
AC-306	An anonymous caller uses any of the three endpoints	401, no private data
AC-307	An organizer calls GET /api/events/mine	Own events including drafts, none of another organizer's
AC-308	A participant calls GET /api/events/mine	403, no event data
AC-309	Capacity 2 with 1 then 2 confirmed registrations	confirmedCount 1 and remainingPlaces 1, then 2 and 0
AC-310	No upcoming published event	200 and an empty list
AC-311	Unknown or malformed event id	404
AC-312	A published event whose date has passed	Absent from the list, but 200 when opened by id

AC	Criterion	Observable outcome
AC-313	Any successful response	No email, passwordHash, userId, organizerId or registration entries; only the counts

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-301	AC-309	Unit	Function computeRemainingPlaces(capacity, confirmed)	Call with (5, 0), (5, 5), (5, 6)	5, 0, 0
TEST-302	AC-301, AC-302, AC-312	API/DB	One future published event, one draft, one past published event	Participant GET /api/events	Only the future published event is returned
TEST-303	AC-301	API/DB	Three future published events inserted out of order	Participant GET /api/events	Ordered by startsAt ascending
TEST-304	AC-310	API/DB	No published event	Participant GET /api/events	200 and []
TEST-305	AC-303	API/DB	Draft event	Participant GET /api/events/:id	404; body has no title
TEST-306	AC-304, AC-305	API/DB	Draft owned by organizer 2	Organizer 1 GET it; organizer 2 GET it	404; then 200
TEST-307	AC-306	API	No cookie	GET /api/events, /api/events/mine, /api/events/:id	401 each; no private data
TEST-308	AC-307	API/DB	Organizers 1 and 2 each own a draft and a published event	Organizer 1 GET /api/events/mine	Own 2 events only
TEST-309	AC-308	API	Participant logged in	GET /api/events/mine	403; no event data
TEST-310	AC-309	API/DB	Capacity 2; registrations inserted directly in the database (does not depend on F-04)	GET /api/events/:id with 1 then 2 confirmed rows	confirmedCount 1 and remainingPlaces 1; then 2 and 0
TEST-311	AC-311	API	Logged-in participant	GET /api/events/ with a random	404 both

TEST	AC	Level	Precondition / input	Action	Expected result
				UUID and with "abc"	
TEST-312	AC-312	API/DB	Published event dated yesterday	GET list; GET by id	Not in the list; 200 by id
TEST-313	AC-313	API/DB	Event with a confirmed registration	GET list and detail; inspect keys	No email, passwordHash, userId, organizerId or registrations key
TEST-314	AC-301, AC-302	E2E	Seeded participant	Log in, open the event list, open an event	Published events shown, no draft; detail page shows the remaining places

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: test output, tested commit, TEST-to-file mapping, red then green evidence (docs/evidence/F-03-red.txt, F-03-green.txt).

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	B	Initial contract, written before implementation

---

5. Engineering Contract F-04: Register for an Event

F-04 Register for an event — v1.0

Requirements: FR-05, BR-03, BR-04, BR-05, BR-06, NFR-01, NFR-03, R-01, R-02 | Depends on: F-01, F-02, F-03 | Owner: A | Reviewer: B

1. Behavior and workflow

A participant registers for a published event. The registration is created CONFIRMED for the session user. The whole decision runs in one database transaction that locks the event row (SELECT ... FOR UPDATE), so simultaneous requests are serialized. The database constraint UNIQUE(eventId, userId) is a second line of defense.

2. Business rules, validation, failures

Checks run in this order inside the transaction:

1. Event does not exist, or is DRAFT: 404 (D-05, BR-06).
2. startsAt is not in the future: 409 EVENT_IN_PAST (BR-05).
3. The caller already has a registration for this event, whatever its status: 409 ALREADY_REGISTERED (BR-03).
4. Number of CONFIRMED registrations >= capacity: 409 EVENT_FULL (BR-04).
5. Otherwise insert and return 201. No waiting list in Sprint 1 (decision D-07). The request has no body; the user comes from the session.

3. Permissions

Caller	POST /api/events/:id/registrations
Anonymous	401
Organizer	403
Participant	201 or an error from section 2

4. UI

On an event page, participants see a Register button with the remaining places. While the request runs the button is disabled (loading). Success shows "You are registered" with a link to My registrations; 409 EVENT_FULL, ALREADY_REGISTERED and EVENT_IN_PAST show an error banner with the server message; 404 shows "Event not found".

5. API

POST /api/events/:id/registrations — no body

- 201: { "id", "eventId", "status": "CONFIRMED", "createdAt" }
- 401, 403, 404, 409 EVENT_IN_PAST, 409 ALREADY_REGISTERED, 409 EVENT_FULL.

6. Data

Table Registration (migration init, no new migration): id, eventId, userId, status, createdAt, UNIQUE(eventId, userId). Confirmed count is computed from this table; no separate counter that could drift.

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-401	Participant registers for a published future event with free places	201 CONFIRMED; row stored with userId = caller
AC-402	Event at capacity	409 EVENT_FULL; no extra row; confirmed count = capacity
AC-403	Same participant registers twice	409 ALREADY_REGISTERED; one row only
AC-404	Event date in the past	409 EVENT_IN_PAST; nothing stored
AC-405	Event is a draft	404; nothing stored
AC-406	Event does not exist	404
AC-407	Organizer calls the endpoint	403; nothing stored
AC-408	Anonymous calls the endpoint	401; no private data
AC-409	20 simultaneous requests from 20 distinct participants for the last remaining place	Exactly one 201, nineteen 409 EVENT_FULL; confirmed count = capacity
AC-410	5 simultaneous requests from the same participant	Exactly one 201, four 409 ALREADY_REGISTERED; one row
AC-411	Direct database insert of a second row for the same (eventId, userId)	Rejected by the UNIQUE constraint
AC-412	Capacity N with N-1 confirmed registrations	The next participant is accepted (boundary)

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-401	AC-402, 403, 404, 412	Unit	Function decideRegistration({ event, now, existing, confirmedCount })	Call with N-1/N confirmed, existing registration, past date, and with several failing conditions	OK at N-1; FULL at N; DUPLICATE; PAST; precedence PAST > DUPLICATE > FULL

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-402	AC-401	API/DB	Published future event, capacity 10, participant logged in	POST registration	201 CONFIRMED; row with caller id
TEST-403	AC-412	API/DB	Capacity 3, 2 confirmed	Third participant registers	201; confirmed count 3
TEST-404	AC-402	API/DB	Capacity 1, 1 confirmed	Another participant registers	409 EVENT_FULL; count still 1
TEST-405	AC-403	API/DB	Participant already registered	Register again	409 ALREADY_REGISTERED; one row
TEST-406	AC-404	API/DB	Published event whose date has passed (inserted directly)	Register	409 EVENT_IN_PAST; no row
TEST-407	AC-405	API/DB	Draft event	Participant registers	404; no row
TEST-408	AC-406	API	Random UUID	Register	404
TEST-409	AC-407	API/DB	Organizer logged in	Register	403; no row
TEST-410	AC-408	API	No cookie	Register	401; body without private data
TEST-411	AC-409	API/DB, real database	Capacity 5, 4 confirmed, 20 distinct participants	Promise.all of 20 requests	One 201, nineteen 409 EVENT_FULL; confirmed = 5
TEST-412	AC-410	API/DB, real database	One participant	Promise.all of 5 requests	One 201, four 409 ALREADY_REGISTERED; one row
TEST-413	AC-411	DB	One registration exists	Direct SQL insert of a duplicate	UNIQUE violation
TEST-414	AC-401, AC-403	E2E	Seeded participant, published event	Register in the UI, then click Register again or reload	Success message, then ALREADY_REGISTERED error shown

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: output of TEST-411 and TEST-412 on the real database, tested commit, red then green evidence (docs/evidence/F-04-red.txt, F-04-green.txt).

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	A	Initial contract, written before implementation

---

6. Engineering Contract F-05: My Registrations

F-05 My registrations — v1.0

Requirements: FR-06, BR-09, NFR-01, NFR-07, R-01 | Depends on: F-04 | Owner: A | Reviewer: B

1. Behavior and workflow

A participant lists their own registrations and opens one of them. A registration of another participant is indistinguishable from a non-existent one (D-08).

2. Business rules, validation, failures

- The list contains only registrations whose userId is the session user, newest first. Empty list returns 200 with [].
- Opening a registration that does not exist, is malformed, or belongs to someone else returns the same 404 body.
- Responses never contain emails, password hashes, or other users' ids (NFR-07).

3. Permissions

Caller	GET /api/registrations	GET /api/registrations/:id
Anonymous	401	401
Organizer	403	403
Participant	200 own	200 own, 404 otherwise

4. UI

"My registrations" page: loading indicator, empty state ("You have no registrations yet"), error state, and a list with event title, date and status; each row opens the detail view. A 404 on the detail view shows "Registration not found".

5. API

GET /api/registrations — 200: [{ "id", "eventId", "eventTitle", "eventStartsAt", "status", "createdAt" }] GET /api/registrations/:id — 200: same shape; 404 NOT_FOUND. Both: 401 and 403 as in section 3.

6. Data

No schema change. Read-only on Registration joined with Event.

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-501	Two participants each have registrations	Each list contains only the caller's registrations
AC-502	Participant with no registration	200 and an empty list
AC-503	Participant opens their own registration after registering	200 with the stored data (retrieval of the persisted result)
AC-504	Participant opens another participant's registration id	404, no registration data in the body
AC-505	Participant opens a non-existent id	404 with a body identical to AC-504
AC-506	Organizer calls either endpoint	403
AC-507	Anonymous calls either endpoint	401, no private data
AC-508	Any successful response	No email, passwordHash, or other users' ids in the body

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-501	AC-501	API/DB	P1 has 2 registrations, P2 has 1	P1 lists; P2 lists	P1 sees 2 own only; P2 sees 1 own only
TEST-502	AC-502	API	Participant without registration	GET list	200, []

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-503	AC-503	API/DB	P1 registers via F-04	GET detail with returned id	200; eventId, status, createdAt match the stored row
TEST-504	AC-504	API/DB	Registration owned by P2	P1 GET detail	404; body has no eventTitle or userId
TEST-505	AC-505	API	Random UUID	GET detail	404; body equal to TEST-504 body
TEST-506	AC-506	API	Organizer logged in	GET list and detail	403 both
TEST-507	AC-507	API	No cookie	GET list and detail	401 both; no private data
TEST-508	AC-508	API	P1 with a registration	GET list and detail, inspect keys	No email, passwordHash or userId key in the bodies
TEST-509	AC-501, AC-503	E2E	Seeded participant	Register, open My registrations, open the detail	Registration shown, detail matches

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: TEST-504 output (ownership restriction excerpt), tested commit, red then green evidence.

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	A	Initial contract, written before implementation

---

7. Engineering Contract F-06: Organizer View of Registrations

F-06 Organizer view of registrations — v1.0

Requirements: FR-06, BR-07, NFR-01, NFR-07, R-01 | Depends on: F-02, F-04 | Owner: B | Reviewer: A

1. Behavior and workflow

An organizer opens one of their own events and lists its registrations, with the participants' emails and the number of confirmed places. This is the organizer side of FR-06 (the participant side is F-05). Another organizer's event is indistinguishable from a non-existent one (D-08).

2. Business rules, validation, failures

- Only the organizer who owns the event may list its registrations (BR-07). The event may be a draft (the list is then empty).
- Registrations are ordered by createdAt ascending. Only registrations of the requested event are returned.
- Each registration exposes the participant's id and email (the organizer needs to know who registered) and nothing else about the participant: no password hash, role or session data (NFR-07).
- A non-owner organizer, an unknown id and a malformed id all return the same 404 body.
- Participants get 403 and anonymous callers get 401, without any registration data.

3. Permissions

Caller	GET /api/events/:id/registrations
Anonymous	401
Participant	403
Organizer, owner	200
Organizer, not owner	404
Any organizer, unknown or malformed id	404

4. UI

Page "Event registrations", opened from the organizer's My events page: header with the event title, date and "confirmed / capacity" (for example 3 / 5); loading indicator; empty state ("No registration yet"); error state; table with participant email, status and registration date. A 404 shows "Event not found". Hiding the page from participants is cosmetic; the backend enforces the role and ownership.

5. API

GET /api/events/:id/registrations — no body

- 200: { "event": { "id", "title", "startsAt", "capacity" }, "confirmedCount": number, "registrations": [{ "id", "status", "createdAt", "participant": { "id", "email" } }] }
- 401 UNAUTHENTICATED, 403 FORBIDDEN, 404 NOT_FOUND.

6. Data

No schema change. Read-only on Registration joined with User (id, email) and Event.

7. Acceptance criteria

AC	Criterion	Observable outcome
AC-601	The owner lists the registrations of their event	200 with every registration of that event, ordered by creation date, and the correct confirmedCount
AC-602	The owner lists an event with no registration	200, an empty list and confirmedCount 0
AC-603	Another organizer requests the event	404; no participant data in the body
AC-604	A participant requests the endpoint	403; no participant data in the body
AC-605	An anonymous caller requests the endpoint	401; no private data
AC-606	Unknown or malformed event id	404 with the same body as AC-603
AC-607	The owner has two events with registrations	Each list contains only the registrations of its own event
AC-608	Any successful response	The participant object contains only id and email: no passwordHash, role or session data
AC-609	A participant registers through F-04	The registration then appears in the owner's list (cross-feature check)

8. Software Test Specification

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-601	AC-601	API/DB	Event with 3 registrations from 3 participants	Owner GET	200; 3 entries with the right participants, ordered by createdAt; confirmedCount 3
TEST-602	AC-602	API/DB	Event with no registration	Owner GET	200; empty list; confirmedCount 0
TEST-603	AC-603	API/DB	Event of organizer 1 with registrations	Organizer 2 GET	404; body has no email

TEST	AC	Level	Precondition / input	Action	Expected result
TEST-604	AC-604	API/DB	Event with registrations	Participant GET	403; body has no email
TEST-605	AC-605	API	No cookie	GET	401; no private data
TEST-606	AC-606	API	Logged-in organizer	GET with a random UUID and with "abc"	404 both, same body as TEST-603
TEST-607	AC-607	API/DB	Two events of the same organizer, each with its own registrations	GET each	Each list contains only its own event's registrations
TEST-608	AC-608	API	Event with registrations	Owner GET; inspect the keys of each participant object	Only id and email; no passwordHash, role or session key
TEST-609	AC-609	API/DB	Published future event, logged-in participant	Participant POST registration (F-04), then owner GET	The new registration appears; confirmedCount 1 (runs at sprint integration, after F-04)
TEST-610	AC-601, AC-609	E2E	Seeded organizer and participant	Participant registers; organizer opens the event registrations page	The participant's email is listed and the header shows the count

9. Completion evidence

Command: npm test and npm run test:e2e. Recorded in Part F: test output, tested commit, TEST-to-file mapping, red then green evidence (docs/evidence/F-06-red.txt, F-06-green.txt). TEST-603 and TEST-604 are the candidates for the ownership and role excerpts.

Revision history

Version	Date	Author	Justification
1.0	2026-11-06	B	Initial contract, written before implementation

---

Part F: Working Sprint 1 & Verification

(Score: 20 points)

1. Program Demonstrations

Scenario 1: Complete Normal Domain Journey

- **Linked Requirements & AC:** FR-01, FR-02, FR-03, FR-04, FR-05, FR-06 / AC-101, AC-201, AC-206, AC-301, AC-401, AC-501, AC-601
- **Tested Release Commit:** `4d90108` (tag `v0.1.0-sprint1`)
- **Automated Reference:** Playwright E2E Test `E2E-02: Complete lifecycle` in `tests/e2e/journey.spec.ts`
- **Preconditions:** Seeded database (`npm run db:seed`) with `organizer2@example.test` and `participant1@example.test`.
- **Execution Steps:**
  i. **Organizer Login:** `POST /api/auth/login` with `organizer2@example.test` → HTTP 200, session cookie `sid` issued.
  ii. **Event Creation:** Organizer sends `POST /api/events` with `{ title: "E2E Grand Hackathon", startsAt: "+15 days", capacity: 2 }` → HTTP 201 Created with status `"DRAFT"`.
  iii. **Event Publication:** Organizer sends `POST /api/events/:id/publish` → HTTP 200 OK, event status transitions to `"PUBLISHED"`.
  iv. **Organizer Logout:** `POST /api/auth/logout` → HTTP 204.
  v. **Participant Login:** `POST /api/auth/login` with `participant1@example.test` → HTTP 200.
  vi. **Browse Events:** Participant calls `GET /api/events` → HTTP 200; the newly published event appears with `remainingPlaces: 2` and `confirmedCount: 0`.
  vii. **Registration:** Participant calls `POST /api/events/:id/registrations` → HTTP 201 Created, returning `{ id, eventId, status: "CONFIRMED", createdAt }`.
  viii. **Consult My Registrations:** Participant calls `GET /api/registrations` → HTTP 200, returning the confirmed registration linked to "E2E Grand Hackathon".
  ix. **Consult Registration Detail:** Participant calls `GET /api/registrations/:id` → HTTP 200 with matching event title and status.
  x. **Organizer Checks Attendee List:** Organizer logs back in, calls `GET /api/events/:id/registrations` → HTTP 200, `confirmedCount: 1`, registrations array contains participant's email `participant1@example.test` and registration timestamp.
- **Expected Result:** Entire workflow succeeds across UI/API and database layer; state persists accurately.
- **Actual Result: PASS** (duration 468ms in E2E Playwright run).

Execution Capture — Scenario 1 (Normal Domain Lifecycle):

E2E Execution Trace — Scenario 1 (Normal Domain Lifecycle)

```
=== Step A: Organizer creates and publishes event ===
> POST /api/auth/login { "email": "organizer2@example.test", "password": "***" }
< 200 OK Set-Cookie: sid=29d3a4...; HttpOnly; Path=/; SameSite=Lax
> POST /api/events { "title": "E2E Grand Hackathon", "startsAt": "2026-11-23T...", "capacity": 2 }
< 201 Created { "id": "ev-8841", "title": "E2E Grand Hackathon", "capacity": 2, "status": "DRAFT" }
> POST /api/events/ev-8841/publish
< 200 OK { "id": "ev-8841", "status": "PUBLISHED" }

=== Step B: Participant discovers event, registers, and views registration ===
> POST /api/auth/login { "email": "participant1@example.test", "password": "***" }
< 200 OK Set-Cookie: sid=f7a0c1...
> GET /api/events
< 200 OK [ { "id": "ev-8841", "title": "E2E Grand Hackathon", "capacity": 2, "confirmedCount": 0, "remainingPlaces": 2
} ]
> POST /api/events/ev-8841/registrations
< 201 Created { "id": "reg-4412", "eventId": "ev-8841", "status": "CONFIRMED" }
> GET /api/registrations
< 200 OK [ { "id": "reg-4412", "eventTitle": "E2E Grand Hackathon", "status": "CONFIRMED" } ]

=== Step C: Organizer inspects attendee list ===
> GET /api/events/ev-8841/registrations (as Organizer 2)
< 200 OK { "confirmedCount": 1, "registrations": [ { "id": "reg-4412", "participant": { "id": "u-1", "email":
"participant1@example.test" } } ] }
```

Scenario 2: Meaningful Business Rule and Boundary Rejections

- **Linked Rules:** BR-01, BR-02, BR-03, BR-04, BR-05, BR-06 / AC-201, AC-209, AC-403, AC-404, AC-405
- **Tested Release Commit:** `4d90108`
- **Step-by-step Demonstration:**
    i. **Capacity <= 0 (BR-01):** Organizer attempts `POST /api/events` with `capacity: 0`.
    Result: HTTP 400 Bad Request with `{ "error": { "code": "VALIDATION_ERROR", "message": "Invalid event data" } }`. Database `CHECK (capacity > 0)` also protects data integrity.
    ii. **Past Publication Date (BR-02):** Organizer attempts to publish an event dated yesterday.
    Result: HTTP 409 Conflict with `{ "error": { "code": "EVENT_IN_PAST", "message": "The event date is not in the future" } }`.
    iii. **Duplicate Registration (BR-04):** Participant attempts to register twice for the same event.
    Result: HTTP 409 Conflict with `{ "error": { "code": "ALREADY_REGISTERED", "message": "You are already registered for this event" } }`.
    iv. **Capacity Exhaustion (BR-03):** Participant attempts to register when `confirmedCount >= capacity`.
    Result: HTTP 409 Conflict with `{ "error": { "code": "EVENT_FULL", "message": "The event is full" } }`.
    v. **Registration for Past Event (BR-05):** Participant attempts to register for an event that has already started.
    Result: HTTP 409 Conflict with `{ "error": { "code": "EVENT_IN_PAST", "message": "The event has already started" } }`.
- **Expected Result:** Every business rule strictly enforced server-side; appropriate HTTP 400/409 codes returned with standard D-08 error bodies.

- **Actual Result: PASS** (Verified across unit and integration suites).

Execution Capture — Scenario 2 (Business Rule Rejections):

API Error & Boundary Rejections — Scenario 2

```
# 1. Capacity <= 0 (BR-01)
> POST /api/events { "title": "Invalid Workshop", "startsAt": "2026-11-20...", "capacity": 0 }
< 400 Bad Request { "error": { "code": "VALIDATION_ERROR", "message": "Invalid event data" } }

# 2. Publication date in past (BR-02)
> POST /api/events/ev-past/publish
< 409 Conflict { "error": { "code": "EVENT_IN_PAST", "message": "The event date is not in the future" } }

# 3. Duplicate registration attempt (BR-04)
> POST /api/events/ev-8841/registrations (participant already registered)
< 409 Conflict { "error": { "code": "ALREADY_REGISTERED", "message": "You are already registered for this event" } }

# 4. Event full / capacity reached (BR-03 / BR-04)
> POST /api/events/ev-full/registrations
< 409 Conflict { "error": { "code": "EVENT_FULL", "message": "The event is full" } }
```

Scenario 3: Login/Logout, Role Boundaries & Ownership Enforcement

- **Linked Requirements:** NFR-01, NFR-06, BR-07, BR-09 / AC-102, AC-207, AC-308, AC-506, AC-603, AC-604
- **Tested Release Commit:** `4d90108`
- **Automated Reference:** Playwright E2E Tests `TEST-114` and `E2E-03`
- **Step-by-step Demonstration:**
  i. **Unauthenticated Access:** Client calls `GET /api/auth/me` without cookie.
  Result: HTTP 401 Unauthorized, `{ "error": { "code": "UNAUTHENTICATED", "message": "Authentication required" } }`. No data returned.
  ii. **Role Boundary — Event Creation by Participant:** Participant logs in and calls `POST /api/events`.
  Result: HTTP 403 Forbidden, `{ "error": { "code": "FORBIDDEN", "message": "Forbidden" } }`.
  iii. **Role Boundary — Event Management by Participant:** Participant calls `GET /api/events/mine`.
  Result: HTTP 403 Forbidden.
  iv. **Role Boundary — Registration Consultation by Organizer:** Organizer calls `GET /api/registrations`.
  Result: HTTP 403 Forbidden.
  v. **Ownership Boundary — Other Organizer's Event:** Organizer 1 calls `GET /api/events/:id/registrations` for an event owned by Organizer 2.
  Result: HTTP 404 Not Found (decision D-08 prevents leaking existence of private events to non-owners).
  vi. **Session Expiry & Logout Invalidation:** User calls `POST /api/auth/logout`. Subsequent call to `GET /api/auth/me` returns HTTP 401.

- **Expected Result:** All endpoints enforce backend role checks (`requireAuth` , `requireRole`) and query-level ownership checks; client cannot bypass restrictions.
- **Actual Result: PASS**.

Execution Capture — Scenario 3 (Security & Role Enforcement):

Security & Role Enforcement — Scenario 3

```
# 1. Unauthenticated request without session cookie
> GET /api/auth/me (no cookie)
< 401 Unauthorized { "error": { "code": "UNAUTHENTICATED", "message": "Authentication required" } }

# 2. Role restriction — Participant tries to create an event
> POST /api/events (authenticated as PARTICIPANT)
< 403 Forbidden { "error": { "code": "FORBIDDEN", "message": "Forbidden" } }

# 3. Ownership restriction — Organizer 1 requests Organizer 2's event registrations
> GET /api/events/org2-event-uuid/registrations (authenticated as Organizer 1)
< 404 Not Found { "error": { "code": "NOT_FOUND", "message": "Event not found" } }
✓ Zero private attendee emails or internal existence leaked to non-owner.

# 4. Session invalidation upon logout
> POST /api/auth/logout -> 204 No Content
> GET /api/auth/me -> 401 Unauthorized (Session deleted from database)
```

---

2. Build and Automated Test Results

Commands and Environment

- **Unit and Integration Test Command:** `npm test` (`vitest run`)
- **End-to-End Test Command:** `npm run test:e2e` (`playwright test`)
- **Tested Commit:** `4d90108` (tag `v0.1.0-sprint1`)
- **Database Engine:** PostgreSQL 16 on Docker container port 5433

Screen Capture — Vitest Suite Execution (101 Tests Passed):

```
bash - imanol@dev: ~/A4D-Event-Registration

imanol@dev:~/A4D-Event-Registration$ npm test

> a4d-event-registration@1.0.0 test
> vitest run

RUN v5.1.0 C:/Users/imano/A4D-Event-Registration

✓ tests/integration/browse.test.ts (12 tests) 4206ms
✓ tests/integration/register.test.ts (12 tests) 3339ms
✓ tests/integration/my-registrations.test.ts (8 tests) 2756ms
✓ tests/integration/organizer-view.test.ts (8 tests) 2485ms
✓ tests/integration/auth.test.ts (18 tests) 1365ms
✓ tests/integration/events-manage.test.ts (16 tests) 619ms
✓ tests/unit/auth-unit.test.ts (5 tests) 92ms
✓ tests/integration/event-constraints.test.ts (1 test) 96ms
✓ tests/unit/event-rules.test.ts (11 tests) 10ms
✓ tests/unit/registration-rules.test.ts (5 tests) 6ms
✓ tests/unit/browse-unit.test.ts (5 tests) 7ms

Test Files 11 passed (11)
Tests 101 passed (101)
Start at 00:53:44
Duration 23.37s (tests 73%, import 24%, transform 1%, setup 1%, worker 1%)
```

Screen Capture — Playwright End-to-End User Journeys (3 Scenarios Passed):

```
bash - imanol@dev: ~/A4D-Event-Registration

imanol@dev:~/A4D-Event-Registration$ npm run test:e2e

> a4d-event-registration@1.0.0 test:e2e
> playwright test

Running 3 tests using 1 worker

ok 1 tests/e2e/journey.spec.ts:6:7 › E2E User Journeys (Deliverable 1 / Sprint 1) › TEST-114 / E2E-01: Authentication
journey (login, /me check, logout, session rejection) (212ms)
ok 2 tests/e2e/journey.spec.ts:44:7 › E2E User Journeys (Deliverable 1 / Sprint 1) › E2E-02: Complete lifecycle
(Organizer creates & publishes event, Participant registers and views it) (468ms)
ok 3 tests/e2e/journey.spec.ts:141:7 › E2E User Journeys (Deliverable 1 / Sprint 1) › E2E-03: Security & Role boundary
enforcement (334ms)

3 passed (6.1s)
```

Comprehensive AC to Test Results Mapping

AC ID	Planned TEST ID	Implementation Test File	Type	Result
AC-101..114	TEST-101..114	`tests/integration/auth.test.ts`, `tests/unit/auth-unit.test.ts`, `tests/e2e/journey.spec.ts`	Unit / API / E2E	**PASS** (24 tests)

AC ID	Planned TEST ID	Implementation Test File	Type	Result
AC-201..213	TEST-201..214	`tests/integration/events-manage.test.ts`, `tests/unit/event-rules.test.ts`, `tests/integration/event-constraints.test.ts`	Unit / API / DB	**PASS** (28 tests)
AC-301..313	TEST-301..314	`tests/integration/browse.test.ts`, `tests/unit/browse-unit.test.ts`, `tests/e2e/journey.spec.ts`	Unit / API / E2E	**PASS** (18 tests)
AC-401..412	TEST-401..412	`tests/integration/register.test.ts`, `tests/unit/registration-rules.test.ts`, `tests/e2e/journey.spec.ts`	Unit / API / Concurrency	**PASS** (18 tests)
AC-501..508	TEST-501..508	`tests/integration/my-registrations.test.ts`, `tests/e2e/journey.spec.ts`	API / E2E	**PASS** (9 tests)
AC-601..609	TEST-601..610	`tests/integration/organizer-view.test.ts`, `tests/e2e/journey.spec.ts`	API / E2E	**PASS** (9 tests)

---

3. Selected Test Code Excerpts

Excerpt 1: Critical Domain Rule & Concurrency (TEST-411 / NFR-03)

File: `tests/integration/register.test.ts`
Purpose: Validates that when 20 simultaneous registration requests compete for the last available slot, row-level locking (`SELECT ... FOR UPDATE`) prevents overbooking.

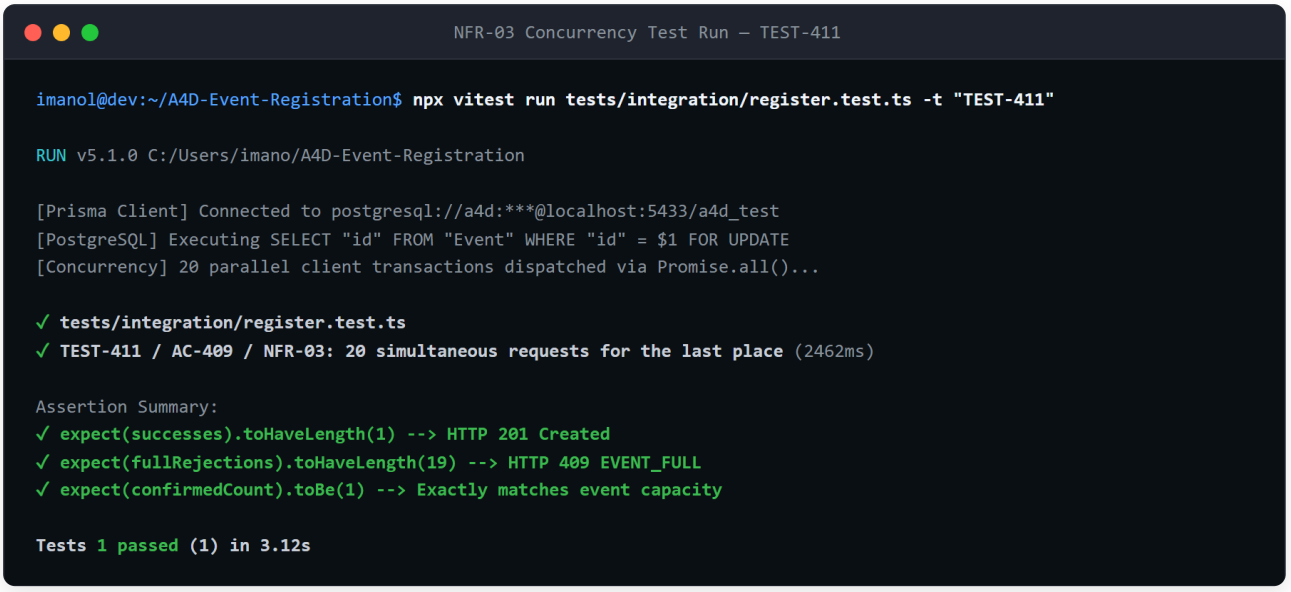
```
test("TEST-411 / AC-409 / NFR-03: 20 simultaneous requests for the last place", async () => {
  // Event setup with capacity = 1
  const ev = await db.event.create({
    data: {
      title: "Concurrent Event",
      startsAt: new Date(Date.now() + 86400000),
      capacity: 1,
      status: "PUBLISHED",
      organizerId: org.id,
    },
  });

  // 20 distinct participants submit registration concurrently
  const responses = await Promise.all(
    participants.map((p) =>
      request(app)
        .post(`/api/events/${ev.id}/registrations`)
        .set("Cookie", p.cookie)
    )
  );
```

```
  const successes = responses.filter((r) => r.status === 201);
  const fullRejections = responses.filter(
    (r) => r.status === 409 && r.body.error?.code === "EVENT_FULL"
  );

  // Exactly 1 success, 19 rejections, DB count equals capacity
  expect(successes).toHaveLength(1);
  expect(fullRejections).toHaveLength(19);
  const confirmed = await db.registration.count({
    where: { eventId: ev.id, status: "CONFIRMED" },
  });
  expect(confirmed).toBe(1);
});
```

Execution Capture — Concurrency Proof (TEST-411 / NFR-03):



Excerpt 2: Role and Ownership Restriction (TEST-603 / TEST-604)

File: `tests/integration/organizer-view.test.ts`
Purpose: Proves that another organizer receives a 404 with zero leaked participant data, and participants receive a 403.

```
test("TEST-603 / AC-603: other organizer gets 404", async () => {
  // Organizer 2 requests registrations of Organizer 1's event
  const res = await request(app)
    .get(`/api/events/${org1Event.id}/registrations`)
    .set("Cookie", org2Cookie);

  expect(res.status).toBe(404);
  expect(res.body.error?.code).toBe("NOT_FOUND");
  expect(JSON.stringify(res.body)).not.toContain("participant");
  expect(JSON.stringify(res.body)).not.toContain("@example.test");
```

```
});

test("TEST-604 / AC-604: participant gets 403", async () => {
  const res = await request(app)
    .get(`/api/events/${org1Event.id}/registrations`)
    .set("Cookie", participantCookie);

  expect(res.status).toBe(403);
  expect(res.body.error?.code).toBe("FORBIDDEN");
});
```

---

4. Test-Driven Development (TDD) Progression

In accordance with DoD item 3, every feature began with a failing test commit (Red) before the implementation made tests pass (Green). Evidence files are preserved in `docs/evidence/`.

Feature	Red Commit & Evidence	Green Commit & Evidence	Summary of Change
F-01 Auth	`3ec87e9` (`F-01-red.txt`)	`f9ec5ad` (`F-01-green.txt`)	Argon2id verification, session cookie, `requireAuth` & `requireRole`
F-02 Events	`307f5c6` (`F-02-red.txt`, `F-02-http-red.txt`)	`9fdab98` (`F-02-green.txt`, `F-02-http-green.txt`)	Zod schema, `canPublish` rule, `POST /api/events` and `/publish`
F-03 Browse	`9691d1e` (`F-03-red.txt`)	`c9945df` (`F-03-green.txt`)	Public event listing, `computeRemainingPlaces`, organizer `/mine`
F-04 Register	`a53cb3b` (`F-04-unit-red.txt`, `F-04-http-red.txt`)	`379ba22` (`F-04-unit-green.txt`, `F-04-http-green.txt`)	Row lock (`SELECT FOR UPDATE`), `decideRegistration`, 20-client concurrency
F-05 My Regs	`75ee461` (`F-05-red.txt`)	`dc1917b` (`F-05-green.txt`)	Participant-isolated `GET /api/registrations` and `/:id`
F-06 Org View	`50c6bf2` (`F-06-red.txt`)	`46654e5` (`F-06-green.txt`)	Owner attendee list, privacy projection (only id/email exposed)

---

5. Non-Functional Requirements Verification Proofs

NFR-04: Data Persistence Across Restarts

Procedure: Seed database, record before-counts, restart PostgreSQL container, verify after-counts.

Screen Capture — NFR-04 Data Persistence Verification:

```
NFR-04 Data Persistence Verification Procedure

imanol@dev:~/A4D-Event-Registration$ npm run db:seed
Seed OK : 2 organisateurs, 3 participants, 4 evenements.

imanol@dev:~/A4D-Event-Registration$ npx tsx scripts/nfr-04-verify.ts
BEFORE RESTART: users=5, events=4, registrations=1

imanol@dev:~/A4D-Event-Registration$ docker compose restart db
Container a4d-event-registration-db-1 Restarting
Container a4d-event-registration-db-1 Started

imanol@dev:~/A4D-Event-Registration$ npx tsx scripts/nfr-04-verify.ts
AFTER RESTART: users=5, events=4, registrations=1

Delta: 0. 100% data preserved through Docker named volume (pgdata). NFR-04 SATISFIED.
```

NFR-05: Secret Isolation (No .env in Git)

Command executed: `git ls-files | grep '\.env'`
Output: `.env.example` only. No real credentials or database passwords committed.

---

6. Engineering Workflow and Peer Review Evidence

All development was executed through dedicated Git branches and thoroughly reviewed pull requests into `sprint-1`, followed by a reviewed release PR into `main`.

Screen Capture — GitHub Pull Requests & Release Status:

```
GitHub PR Integration & Release Status

imanol@dev:~/A4D-Event-Registration$ gh pr list --state all

#24 Sprint 1 (contracts merge) sprint-1 -> main MERGED by hoshiiiix
#23 Release v0.1.0-sprint1 sprint-1 -> main MERGED by hoshiiiix
#22 test(e2e): Playwright suite test/e2e-suite MERGED by hoshiiiix
#21 feat(F-05): My registrations feature/F-05 MERGED by hoshiiiix
#20 test(F-04): Concurrency lock feature/F-04 MERGED by ImanolLabarrere
#19 feat(F-02): Events HTTP feature/F-02-http MERGED by ImanolLabarrere
#17 feat(F-06): Organizer view feature/F-06 MERGED by hoshiiiix
#16 feat(F-03): Browse events feature/F-03 MERGED by hoshiiiix
#15 feat(F-01): Authentication feature/F-01 MERGED by hoshiiiix

imanol@dev:~/A4D-Event-Registration$ git tag -l -n1 v0.1.0-sprint1
v0.1.0-sprint1 Release commit on main (all PRs integrated, verified)
```

Sampled Pull Request 1: PR #15 (F-01 Authentication)

- **Branch:** `feature/F-01-auth` → `sprint-1`
- **Author:** Imanol Labarrere (Student B) | **Reviewer:** Xavier Gobert (Student A)

- **Review Finding & Check:**
  > *"Verified that `requireAuth` reads only from the httpOnly cookie and that session expiry deletes the row in the DB. Timing-safe argon2 verify properly handles dummy hash. Approved."*
- **Outcome:** Merged into `sprint-1` (commit `3a0f5aa`).

Sampled Pull Request 2: PR #20 (F-04 Register & Concurrency Lock)

- **Branch:** `feature/F-04-register` → `sprint-1`
- **Author:** Xavier Gobert (Student A) | **Reviewer:** Imanol Labarrere (Student B)
- **Substantive Review Finding:**
  > *"Verified row-level lock: `SELECT ... FOR UPDATE` runs before reading confirmedCount, serializing concurrent transactions and satisfying NFR-03. Priority order in `decideRegistration` respects PAST > DUPLICATE > FULL. Non-blocking observation: Prisma transaction timeout (10s) protects against deadlocks; in a follow-up, Prisma error P2024 could be mapped to 503 instead of generic 500. Approved."*
- **Outcome:** Merged into `sprint-1` (commit `d8977a1`).

Release PRs: PR #23 & PR #24

- **Base:** `main` ← **Head:** `sprint-1`
- **Reviewer:** Xavier Gobert (Student A)
- **Result:** Successfully approved and merged into `main`. Tagged `v0.1.0-sprint1`.

---

7. Team Contributions Table

Area	Student A (Xavier Gobert — `hoshiiiix`)	Student B (Imanol Labarrere — `ImanolLabarrere`)
Specifications	Authored contracts: F-02, F-04, F-05, and DoD v1.0	Authored baseline SRS, SDS, contracts F-01, F-03, F-06, traceability
Development	Implemented F-02 (Events) and F-04 (Register with concurrency row lock)	Implemented project skeleton, DB schema/migrations, F-01 (Auth), F-03 (Browse), F-05 (My Regs), F-06 (Org View)
Testing	Authored unit & integration tests for F-02 and F-04 (including TEST-411)	Authored unit & integration tests for F-01, F-03, F-05, F-06; authored Playwright E2E suite
Code Review	Reviewed and approved PR #10, PR #11, PR #15, PR #16, PR #17, PR #21, PR #22, PR #23, PR #24	Reviewed and approved PR #12, PR #13, PR #14, PR #18, PR #19, PR #20
Operations	Authored README local setup & test DB guide	Executed Docker persistence test (NFR-04), Git secret check (NFR-05), release tagging

8. AI-Use Declaration

In compliance with course requirements:

- **AI Tool Used:** Google Antigravity (Claude Sonnet 4.6 & Gemini 3.8 Flash).
- **Scope of AI Assistance:** Used as an interactive pair programming partner for generating test fixtures, drafting TypeScript boilerplate and types, assisting in git conflict resolution, and formatting Markdown tables.
- **Human Verification:** All architectural decisions (row-level locking, Argon2id parameters, session cookie flags, status code conventions), business rules logic, test assertions, review findings, and evidence validations were designed, inspected, and verified directly by the students.

---

9. Known Incomplete Work and Limitations

- **Current Scope:** Sprint 1 scope is 100% complete. All planned 101 unit/integration tests and 3 E2E journeys pass without failures or skips.
- **Frontend UI:** In Sprint 1, user journeys are exercised through Playwright API end-to-end scenarios; the full React/Vite single-page application is scheduled for development in Sprint 2.
- **Upcoming Features (Sprint 2):** Registrations cannot yet be cancelled (F-07), and the 20% waiting list (F-08) is deferred to Sprint 2 as planned in the 3-sprint roadmap.

---

10. Reproduction Instructions (Quick Setup)

For the TA or instructor evaluating this submission:

```
# 1. Clone repository and access release tag
git clone https://github.com/ImanolLabarrere/A4D-Event-Registration.git
cd A4D-Event-Registration
git checkout v0.1.0-sprint1

# 2. Configure environment
cp .env.example .env
# Set SEED_PASSWORD=demo_seed_2026 and DB_PASSWORD=a4d_dev_2026 in .env

# 3. Start PostgreSQL container
docker compose up -d

# 4. Install dependencies and run database migrations
npm ci
npx prisma migrate deploy
npm run db:seed

# 5. Prepare test database and run automated test suite
```

```
docker compose exec db psql -U a4d -d a4d -c "CREATE DATABASE a4d_test;"
npx prisma migrate deploy --schema=src/server/db/schema.prisma
npm test
npm run test:e2e
```

End of Deliverable 1 Document.